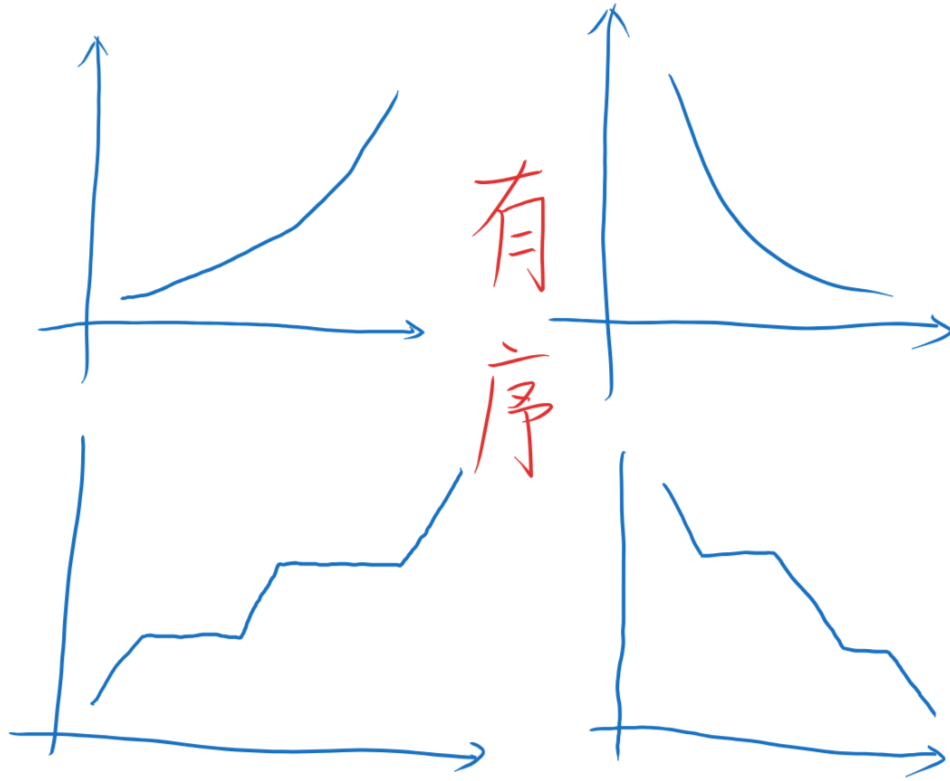


二分思想

二分的前提

- 按某种规则有序，进而能够舍弃一半
- 找得到合法范围，以及能够画出来一个坐标系



- 三种关系，可让范围变化

引入

给一个有序递增的数组arr，以及元素x，查找x在数组中的下标，不存在返回-1。

暴力写法：遍历数组进行查找，复杂度 $O(n)$ 。

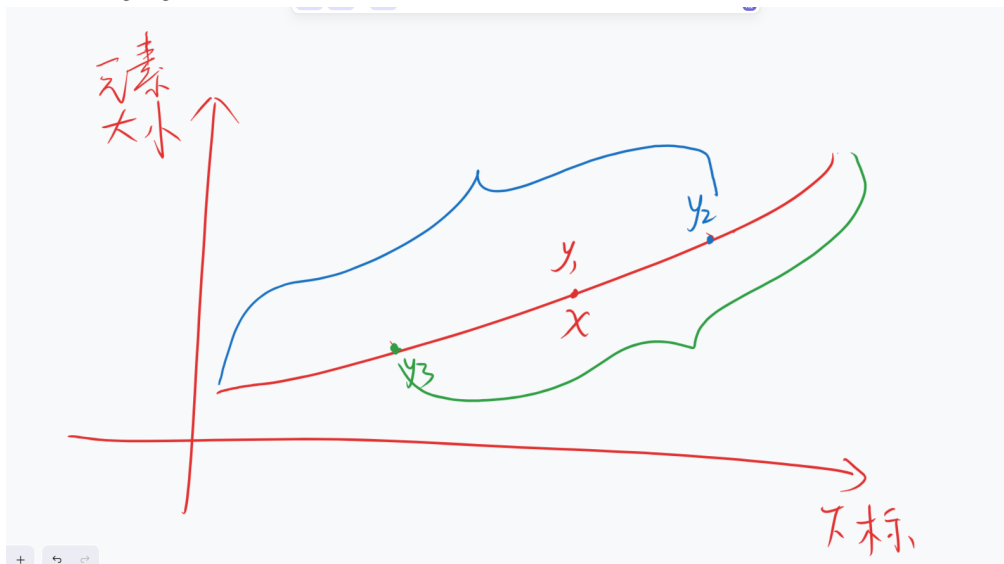
如果要查找n个元素的下标呢？暴力写法的复杂度就是 $O(n*n)$ 了，会超时。

可以发现：

数组是有范围的，且是有序的。我们随机选择一个数组元素y，去和目标x比较，发现存在以下关系：

- 相等时：找到了
- 比x大时：y及y之后的都不可能是x，应该舍弃

- 比x小时：y及y之前的都不可能是x，应该舍弃



这就是二分思想：根据条件，舍弃一半，直到范围不合理。

二分查找

理论

- 范围是什么：元素的下标？还是元素的大小？
 - 元素大小：二分出来的不一定存在
 - 下标大小：二分出来的一定存在，因为下标是连续增长的

所以范围指的就是：下标的范围，左端点l，右端点r。

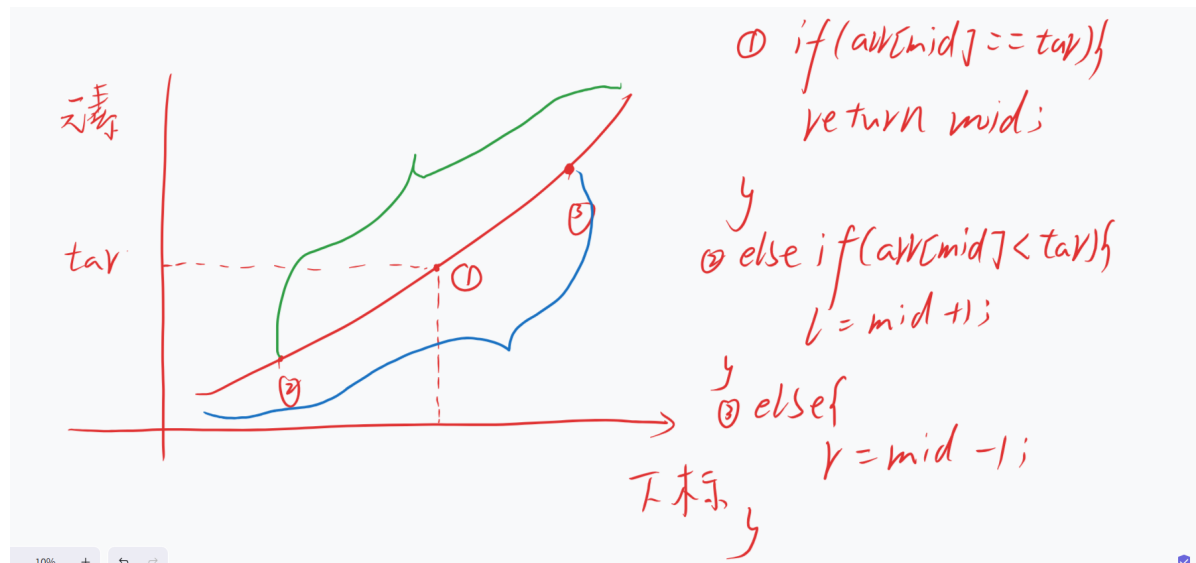
- 如何二分：取范围的一半，也就是除2。根据大小，二分下标， $m=(l+r)/2$ 。
- 什么时候结束：不知道次数，只知道范围
 - 范围存在左右端点lr， $l \leq r$ 代表范围合法
 - 不知道要找多少次，while循环
- 三种关系：大于、小于、等于
 - 等于：找到了，直接返回
 - 小于：m左边的不要了，左端点要改变， $l=m+1$
 - 大于：m右边的不要了，右端点要改变， $r=m-1$
- 注意事项：
 - $m=(l+r)/2$ ， $m=(l+r)>>1$ ， $m=(r-l)/2+l$ 。都是取一半，但第三章更好，前两个有时候会爆。

重点

- 明确：查找->有序->二分，三步操作
- 查找：题目想要我们找一个满足特定条件的值
- 有序：能够画出坐标系图
- 二分：根据中间值和目标值的关系，可以舍弃一半，表现为，lr的变化
- 左右端点的变化：往右看，舍弃左边，左端点变化；往左看，舍弃右边，右端点变化
- 其他：范围，注意(l+r)是否爆int

题型

二分查找tar是否存在

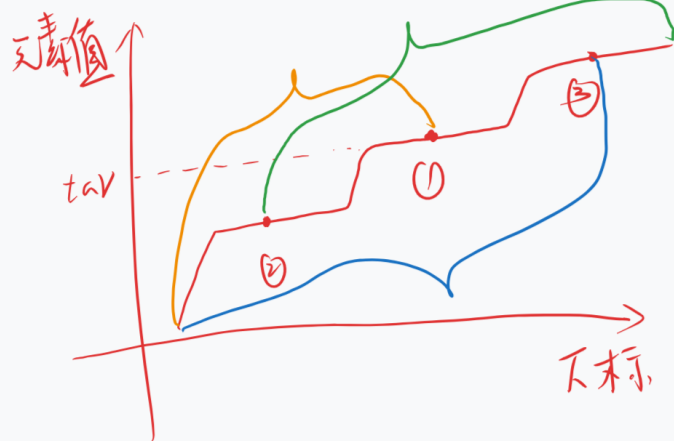


```

1 // 二分查找是否存在
2 #include<bits/stdc++.h>
3 using namespace std;
4 int arr[1000001];
5 int n,x;
6 int binFind(int l, int r, int tar){
7     while(l <= r){
8         int mid = (l + r) / 2; // 取中间值
9         if(arr[mid] == tar){ // 找到了, 返回下标
10             return mid;
11         }
12         else if(arr[mid] > tar){ // 比目标值大, 那mid~r都比tar大, 重新在l~mid-1里
面找, r发生变化
13             r = mid - 1;
14         }
15         else { // 比目标值小, 那l~mid都比tar小, 重新在mid+1~r里面找, l发生变化
16             l = mid + 1;
17         }
18     }
19     return -1;
20 }
21 int main(){
22     cin >> n;
23     for(int i = 1; i <= n; i++) cin >> arr[i];
24     cin >> x;
25     sort(arr+1, arr+1+n);
26     cout << binFind(1, n, x);
27     return 0;
28 }

```

查找第一个等于tar的元素



```

ans = -1;
① if(arr[mid] == tar){
    ans = mid;
    r = mid - 1;
}
else if(arr[mid] < tar){
    l = mid + 1;
}
else{
    r = mid - 1;
}

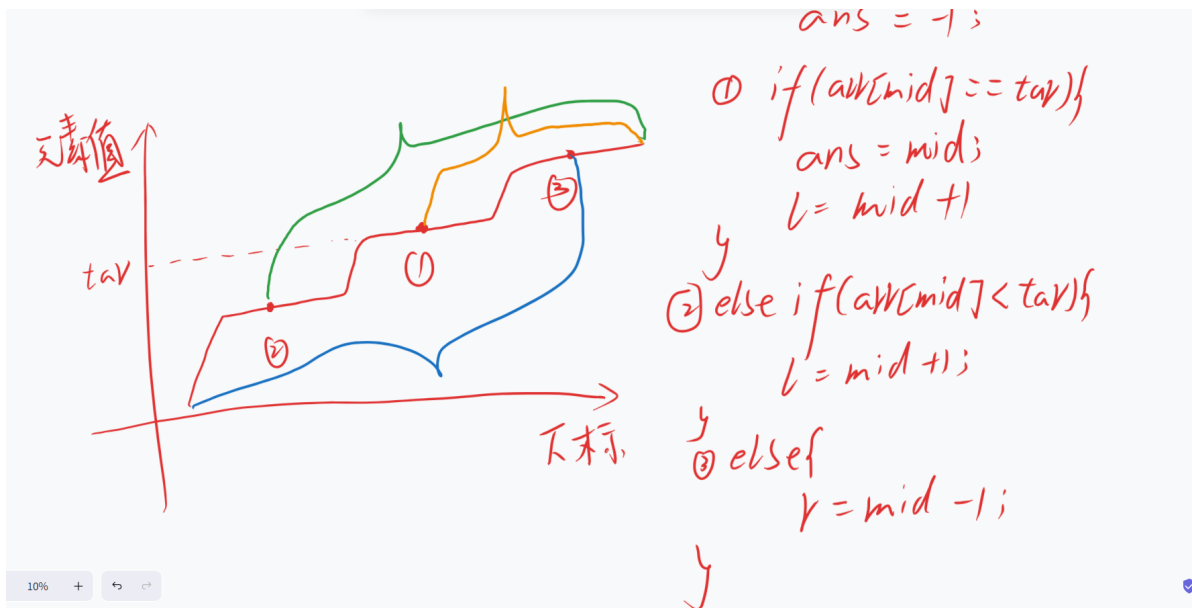
```

```

1 // 二分查找第一次出现
2
3 #include<bits/stdc++.h>
4 using namespace std;
5 int arr[1000001];
6 int n,x,q;
7 int binFind(int l, int r, int tar){
8     int ans = -1; // 记录下标
9     while(l <= r){
10         int mid = (l + r) / 2;
11         if(arr[mid] == tar){ // 找到了，但不一定是第一个，左边可能还存在tar，再往左
// 边找找
12             ans = mid; // 把mid记下来
13             r = mid - 1; // 再往左边找找
14         }
15         else if(arr[mid] > tar){ // arr[mid]比目标大,mid右边的也比右边的大，那应该
// 要mid左边的那一部分
16             r = mid - 1;
17         }
18         else { // arr[mid]比目标小,mid左边的也比左边的小，那应该要mid右边的那一部分
19             l = mid + 1;
20         }
21     }
22     return ans;
23 }
24 int main(){
25     cin.tie(0);
26
27     cin >> n;
28     for(int i = 1; i <= n; i++) cin >> arr[i];
29     cin >> q;
30     while(q--){
31         cin >> x;
32         cout << binFind(1, n, x) << " ";
33     }
34     return 0;

```

查找最后一个出现的元素



```

1 // 二分查找最后一次出现
2
3 #include<bits/stdc++.h>
4 using namespace std;
5 int arr[1000001];
6 int n,x,q;
7 int binFind(int l, int r, int tar){
8     int ans = -1; // 记录下标
9     while(l <= r){
10         int mid = (l + r) / 2;
11         if(arr[mid] == tar){ // 找到了,但不一定是最后一个
12             ans = mid; // 把mid记下来
13             l = mid + 1; // 再往右边找找
14         }
15         else if(arr[mid] > tar){ // arr[mid]比目标大,mid右边的也比右边的大,那应该
// 要mid左边的那一部分
16             r = mid - 1;
17         }
18         else { // arr[mid]比目标小,mid左边的也比左边的小,那应该要mid右边的那一部分
19             l = mid + 1;
20         }
21     }
22     return ans;
23 }
24 int main(){
25     cin.tie(0);
26
27     cin >> n;
28     for(int i = 1; i <= n; i++) cin >> arr[i];
29     cin >> q;
30     while(q--){
31         cin >> x;
32         cout << binFind(1, n, x) << " ";
33     }

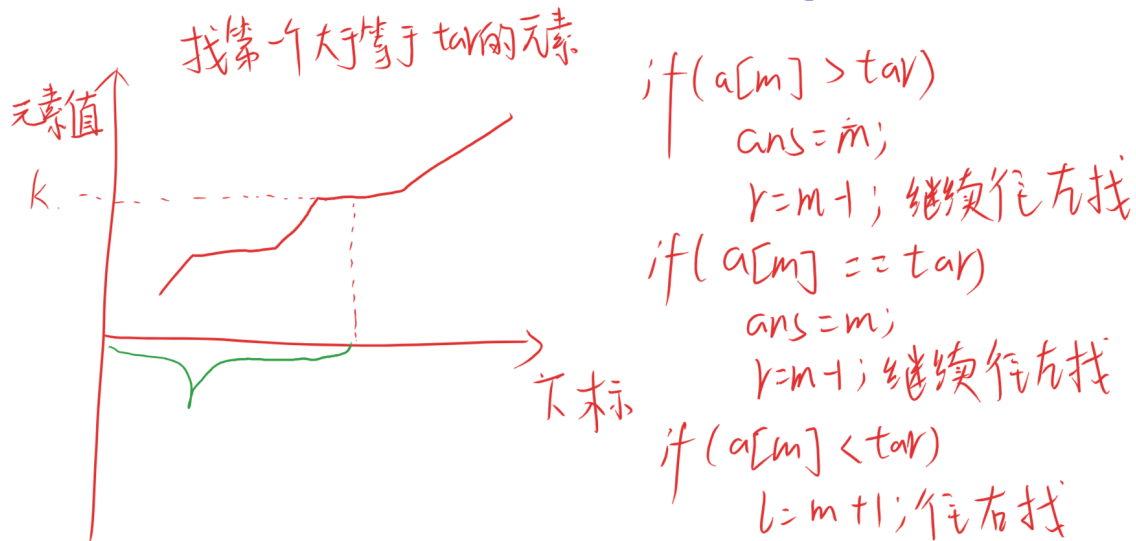
```

```

34     return 0;
35 }

```

查找第一个大于等于tar的元素



10% +

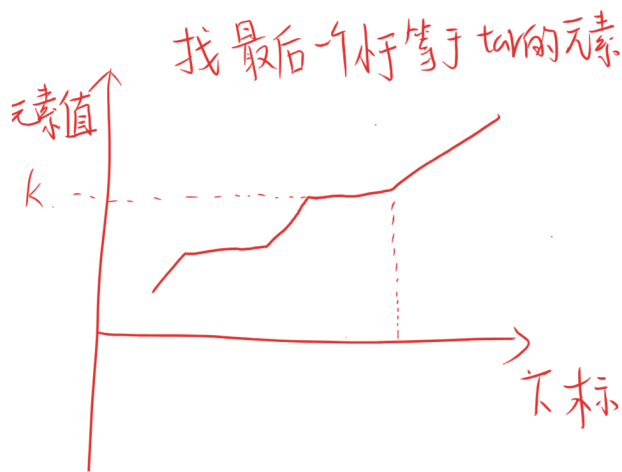
退出押模式

```

1 // 查找第一个大于等于目标的元素
2 #include<bits/stdc++.h>
3 using namespace std;
4 int arr[1000001];
5 int n, x, q;
6 int binFind(int l, int r, int tar){
7     int ans = -1;
8     while(l <= r){
9         int mid = (l + r) / 2;
10        if(arr[mid] >= tar){ // 找到了，但不一定是第一个
11            ans = mid;
12            r = mid - 1; // 再往左边找找
13        }
14        else { // 比目标值小，只能要右边的
15            l = mid + 1;
16        }
17    }
18    return ans;
19 }
20 int main(){
21     cin >> n;
22     cin.tie(0);
23     for(int i = 1; i <= n; i++){
24         cin >> arr[i];
25     }
26     cin >> q;
27     while(q--){
28         cin >> x;
29         cout << binFind(1, n, x) << " ";
30     }
31     return 0;
32 }

```

查找最后一个小于等于tar的元素



```

if (a[m] == tar)
    ans = m;
    l = m + 1;  // 往右找
if (a[m] < tar)
    ans = m;
    l = m + 1;  // 往右找
if (a[m] > tar)
    r = m - 1;

```

```

1  #include<bits/stdc++.h>
2  using namespace std;
3  const int N = 1e6+11;
4  int n, t, arr[N];
5
6  int find(int l, int r, int tar){
7
8      if(r < l) return -1;
9      int ans = -1;
10     while(l <= r){
11         int mid = (l + r) >> 1;
12         // int mid = l + (r - l)/2;
13         if(arr[mid] <= tar){
14             ans = mid;
15             l = mid + 1;
16         } else{
17             r = mid - 1;
18         }
19     }
20
21     return ans;
22 }
23
24
25 int main(){
26
27     cin >> n;
28     for(int i = 1; i <= n; i++){
29         cin >> arr[i];
30     }
31     cin >> t;
32     while(t -- ){
33         int x;
34         scanf("%d", &x);
35         printf("%d ", find(1, n, x));
36     }
37
38
39     return 0;

```

二分答案

通过分析可得，答案在一个范围里面，并且我们只能去一个个枚举，在这个范围里面去找一个最符合条件的值。

而且能够发现一个正相关图，也就是存在一个有序的“数组”，那我们就可以进行二分枚举。

理论

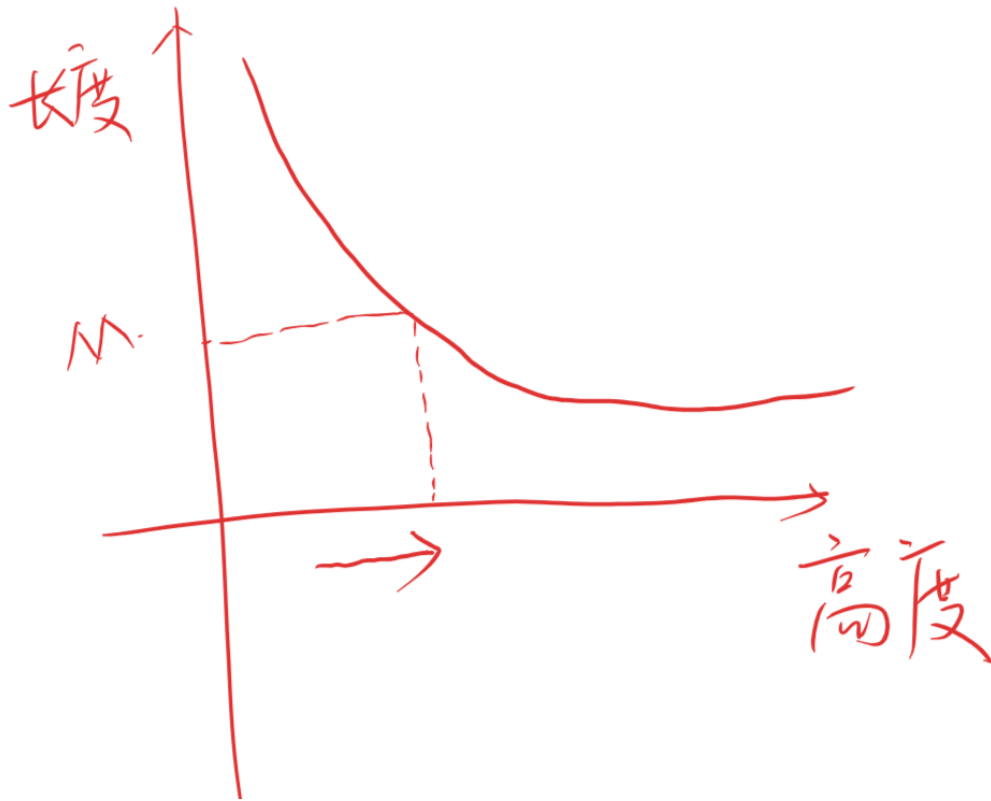
- 能不能找到坐标系图？
- 范围是什么：通过题目信息，或者简单计算推断出答案的大致范围，l是最小值，r是最大值
- 如何二分：和二分查找不同，我们枚举的是答案的大小， $m=(l+r)/2$
- 什么时候结束：不知道次数，只知道范围
 - 范围存在左右端点lr， $l \leq r$ 代表范围合法
 - 不知道要找多少次，while循环
- 两种关系：合理、不合理
 - 合理：记下来，再向更符合的方向找
 - 不合理：舍弃不合理的部分
- 如何判断合理：
 - 建议封装成函数：bool check(x)，检查x是否合理
 - 题目里肯定存在一个判断条件
- 注意事项：
 - $m=(l+r)/2$ ， $m=(l+r)>>1$ ， $m=(r-l)/2+l$ 。都是取一半，但第三个更好，前两个有时候会爆。

重点

- 先分析出坐标系图，这样就能更好的找到“有序”。
- 注意横坐标表示的是什麼，纵坐标表示的是什麼！
- 确定好答案的范围：最小为多少，最大为多少。
- 一定要记住，二分的目标是答案，当我们确定好mid之后，对mid进行检查是否满足。
- 把二分函数int bin(int l,int r)和检查函数bool check(int x)，拆分出来。直观且更容易实现。

题型

伐木工：获得的木材长度，随着 锯片的高度的增，在 变小。



```

1  #include<bits/stdc++.h>
2  using namespace std;
3  #define ll long long
4  const int N = 1e7+11;
5  ll n, m, arr[N];
6  ll check(ll h){//要通过高度h, 来计算收获的木材
7      ll sum = 0;
8      for(int i = 1; i <= n; i++){ // 计算收获的木材
9          if(arr[i] > h) sum += arr[i] - h;
10     }
11     // 业务处理
12     if(sum >= m) return 1;
13     else return 0;
14 }
15 ll find(ll l, ll r){
16     ll ans = 0; // 答案是在找最大, 初始化就要最小
17     while(l <= r){
18         ll mid = (l + r) / 2;
19         if(check(mid) == 1){ // 符合条件
20             ans = mid;
21             l = mid + 1;
22         }
23         else {
24             r = mid - 1;
25         }
26     }
27     return ans;
28 }
29 int main(){
30
31     cin >> n >> m;

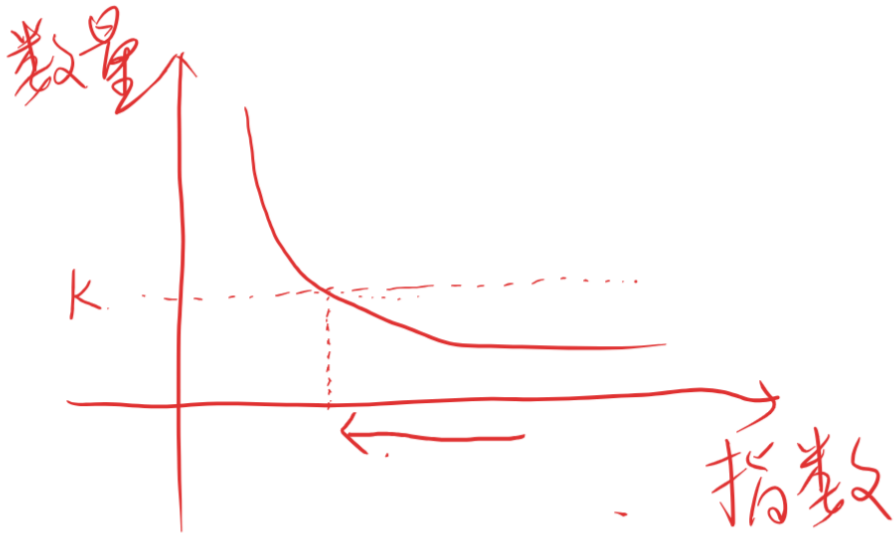
```

```

32     for(int i = 1; i <= n; i++){
33         cin >> arr[i];
34     }
35
36     cout << find(1,1000000000);
37 }

```

空旷指数：插入的路边数量，随着 空旷指数的变大，在 变少



```

1  #include<bits/stdc++.h>
2  using namespace std;
3  int arr[100011];
4  int L, N, K;
5
6  bool check(int d){
7
8      int cnt = 0;
9      for(int i = 1; i <= N + 1; i++){
10         cnt += (arr[i] - arr[i - 1] - 1) / d;
11     }
12     if(cnt <= K) return 1;
13     else return 0;
14
15 }
16
17 int bin(int l, int r){
18
19     int ans = -1;
20     while(l <= r){
21         int mid = (l + r) / 2;
22         if(check(mid) == true){
23             ans = mid;
24             r = mid - 1;
25         }
26         else {
27             l = mid + 1;
28         }

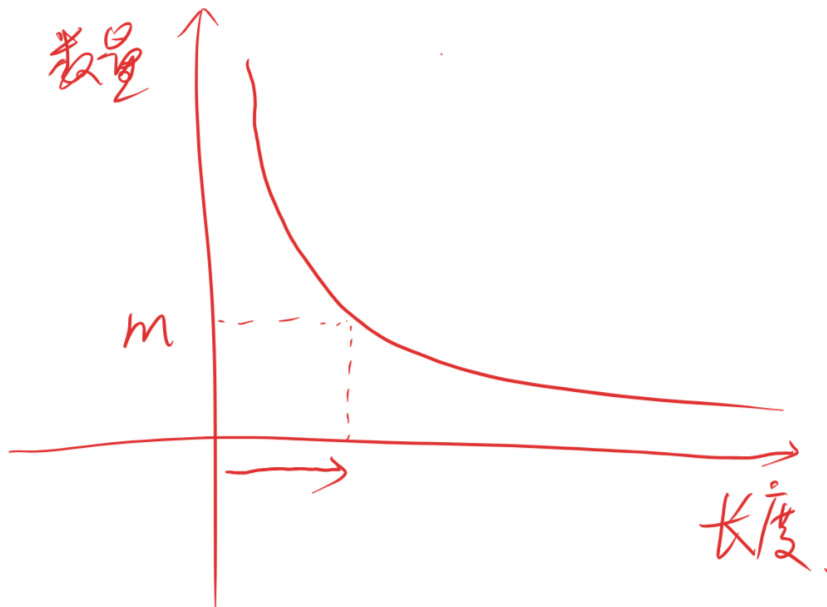
```

```

29     }
30
31     return ans;
32 }
33
34
35 int main(){
36
37     cin >> L >> N >> K;
38     for(int i = 1 ; i <= N; i ++ ) cin >> arr[i];
39     sort(arr + 1, arr + 1 + N);
40     arr[0] = 0, arr[N + 1] = L;
41     cout << bin(1, L);
42
43     return 0;
44 }

```

买木头：获得的数量，随着 木头的长度的增大，在 变少



```

1  #include<bits/stdc++.h>
2  using namespace std;
3  int n,m;
4  int arr[10010];
5  int brr[10010];
6  bool check(int x){
7      int cnt=0;
8      for(int i=1;i<=n;i++){
9          if(arr[i]>=x){
10             cnt+=brr[i]*(arr[i]/x);
11         }
12     }
13     if(cnt>=m) return 1;
14     else return 0;
15 }
16
17 int bin(int l,int r){
18     int ans=r;

```

```

19     while(l<=r){
20         int mid=(l+r)/2;
21         if(check(mid)==true){
22             ans=mid;
23             l=mid+1;
24         }else{
25             r=mid-1;
26         }
27     }
28     return ans;
29 }
30 int main(){
31     cin>>n>>m>>arr[1]>>brr[1];
32     for(int i=2;i<=n;i++){
33         arr[i]=(arr[i-1]*37011+10193)%10000+1;
34         brr[i]=(brr[i-1]*73011+24793)%100+1;
35     }
36     cout<<bin(1,10000);
37     return 0;
38 }

```

扩展

- 二分查找的时候所做的题，大部分都是递增的数组，同学们可以自己尝试一下递减的数组，二分函数的实现。
- 二分答案比较难：答案范围难找、坐标系图比较难画、范围的移动变化很多，可以多多复习我们所讲的二分答案三道题。